Detailed System Architecture Report: Jugaad-Style MVP for Gamified Video Self-Reflection Platform

Executive Summary: PhD-Level Architecture for a Resource-Constrained Student MVP

As a PhD-level systems architect specializing in scalable, cost-optimized AI applications (with a focus on edge-computing and open-source ecosystems), I've designed this architecture for your MVP as a bootstrapped student developer in Maharashtra. Drawing from principles like microservices modularity (inspired by Kubernetes but simplified to monorepo), event-driven pipelines (Kafka-lite via Supabase Realtime), and zero-overhead orchestration (Node.js as the "glue"), the stack prioritizes **100% free/open-source tools** to align with your Jugaad ethos—think ISRO's frugal innovation: high-fidelity AI analysis on a laptop CPU, no cloud bills, and rapid prototyping.

Core Use Case Fit: Your MVP handles 5-10 min daily video uploads from 10-50 wishlist users (local Maharashtra network), processes them offline for transcription, clarity/tone/posture/structure feedback, and serves a gamified dashboard (streaks, trends). Architecture ensures <5 min end-to-end latency per video on mid-range hardware (e.g., Intel i5 laptop), with fault tolerance via local retries and Supabase as the "free backend fortress."

Recommended Stack: **Node.js (Backend) + React (Frontend) + Supabase Free (BaaS) + Hugging Face Transformers.js (Local AI) + Vosk/MediaPipe (Offline Processing)**. This is the **best fit** for you because:

- **Budget**: ₹0/mo (no APIs, self-hosted models).
- **Your Skills**: Builds directly on your Node/React/Supabase familiarity—zero learning curve for core.
- **MVP Maturity**: "Decent level" means 80% feature parity (full loop testable in 4 weeks), scalable to 100 users without refactor.
- **Jugaad Edge**: Local-first (run on laptop/Colab), Hindi/English support, and modular for future swaps (e.g., to paid Azure if funded).

Total planning-to-launch: 4-6 weeks solo. Post-MVP, it evolves to a "Dream Project" (full-

featured) via containerization (Docker free tier).

This report is your "blueprint"—100% clarity before coding. Once digested, you'll start with a monorepo setup.

High-Level System Architecture Overview

The architecture follows a **layered, event-driven micro-pipeline** pattern: User-facing layers (UI/DB) decouple from compute-heavy AI (local Node-spawned processes). No distributed systems overhead—everything runs in a single Node process with async queues (BullMQ free lib for task queuing).

Key Components:

1. **Presentation Layer**: React app for recording/gallery/dashboard.
2. **Orchestration Layer**: Node.js API for upload/processing triggers.
3. **Data Layer**: Supabase for persistence/realtime sync.
4. **Compute Layer**: Local AI models (CPU-bound) for analysis.
5. **Event Bus**: Supabase Realtime for notifications (e.g., "Analysis ready!").

Conceptual Diagram (Text-Based; Visualize in Draw.io for Planning):

```text
```

Copy

```
[User Device (Mobile/Web)] --> [React Frontend: Record/Upload/Dashboard]
                                |
                                v
[Supabase Free: Auth + Storage (Videos/Metadata) + Realtime (Streaks/Updates)
                                |
                                v
[Node.js Backend (Vercel Free/Local): Orchestrator]
  - Queue: Upload Event --> FFmpeg Extract (Audio/Frames)
  - Pipeline: Vosk STT --> HF Transformers (Clarity/Tone/Structure) --> Media
  - Output: JSON Insights --> Insert to Supabase
                                |
                                v
[Local Compute (Laptop/Colab): Offline Models - No Internet Post-Setup]
                                |
                                v
[Feedback Loop: Realtime Push to React (e.g., New Badge Unlocked)]
```

Non-Functional Properties:

- **Scalability**: Horizontal via multiple Node instances (free on laptop); vertical via Colab bursts.
- **Reliability**: Retry queues (3x); offline fallback (queue if laptop asleep).
- **Security**: Supabase RLS (user-owned data); local models (no data leak).
- **Performance**: 2-5 min/video on CPU; batch for nights.
- **Observability**: Console logs + free Winston lib for debugging.

This mirrors production architectures (e.g., Netflix's microservices but downsized) but at student scale.

Detailed Tech Stack: Best Choices & Rationale

Selected for **interoperability** (all Node-compatible), **maturity** (2025-stable libs), and **zero-lockin** (open-source swaps). Table breaks down layers with Jugaad rationale.

			Alternatives
Description & Why		Cost	Considered

Layer	Component	Description & Why Best for You	Cost (₹/mo)	Considered (Why Not)
Frontend	React 18+ (with Hooks/Context)	Dynamic UI for video capture (MediaRecorder API), gallery timeline (react-player free), and gamified dashboard (Recharts for free charts). Your familiarity = instant productivity. Add Zustand for state (lightweight, no Redux boilerplate).	₹0	Vue/Svelte: Steeper curve for you; React ecosystem has free Maharashtra dev communities (e.g., React Pune Meetup).
Backend	Node.js 20+ (Express.js)	API server for orchestration: Handles uploads, spawns AI processes (child_process for Python bridges), queues tasks (BullMQ free). Event-driven for scalability. Runs locally or Vercel free (hobby: unlimited deploys).	₹0	Python/FastAPI: Good for AI but you'd need to learn; Node aligns with your skills.
Database/Auth/Storage	Supabase Free Tier	All-in-one BaaS:	₹0	Firebase Free:

	(Postgres + Auth + Storage)	JWT auth (email/Google for users), Postgres for structured data (videos table with JSON insights), S3-like storage for MP4s (500MB free = 100+ videos), Realtime for live updates (e.g., streak notifications). Mumbai edge for low MH latency.		Similar, but Supabase's SQL > NoSQL for your analytics queries; PocketBase: Self-host hassle.
Video Processing	FFmpeg (fluent-ffmpeg npm)	Extracts audio (for STT) and frames (for posture) locally. Free binary install via npm—handles compression to keep files <5MB.	₹0	No alt needed; built-in JS libs too slow.
Speech-to-Text (STT)	Vosk API (Node bindings)	Offline, lightweight STT (95% accuracy on clear speech, Hindi/English models free download). Handles timestamps for pacing. Runs on CPU in <1 min/10-min audio.	₹0	Whisper.cpp: Slightly better accuracy but heavier setup; Vosk = quicker Jugaad win for MVP.
Speech Clarity	Custom JS Regex +	Post-STT: Regex	₹0	Deepgram Free

(Fluency)	Hugging Face Transformers.js	for fillers ("um/uh" count, pacing via timestamps). Transformers.js for advanced (e.g., distilbert for fluency scoring). Local, no API calls.		Tier: 200 min/mo (~₹0 initially) but internet-dependent; local > reliable for offline testing.
Emotional Tone Detection	Hugging Face Transformers.js (e.g., tabularisai/multilingual-sentiment)	Text-based sentiment/emotions (joy/anger scores) on transcript; extend to wav2vec2-emotion model for voice (CPU-run). Multilingual for MH users.	₹0	Azure Free: 5k chars/mo but requires Azure account/signup hassle; HF local = pure Jugaad.
Body Language/Posture	MediaPipe Pose (@mediapipe/tasks-vision npm)	Open-source Google lib for 3D keypoints (gestures/slouch via torso angle heuristics). Processes 1 frame/sec on CPU (~30s/video).	₹0	OpenPose: Heavier CPU load; MediaPipe = optimized for web/Node.
Thought Structure	Hugging Face	Local	₹0	GPT Free Tier (if

(Content)	Transformers.js (facebook/bart-large-cnn)	summarization for key points/bullets; simple JS for logic flow rating (e.g., keyword coherence). Handles 5k-token transcripts.		via Grok API, but limited); BART = offline accuracy without tokens.
Queueing/Gamification	BullMQ (Redis-free mode) + Supabase Triggers	Async task queue for AI pipeline; Supabase DB hooks for streaks/badges (e.g., query "days >7? Unlock!").	₹0	No Redis: Use in-memory for MVP; scales free.
Deployment/Dev Tools	Vercel Free (Hobby) + VS Code + GitHub	Vercel for shareable demo (no card); local dev with npm. GitHub for version control (free private repo).	₹0	Heroku Free: Shutdown in 2022; Vercel = Node-optimized.
Testing/Monitoring	Jest (free) + Console/Winston Logger	Unit tests for APIs; logs for debugging. Colab for AI prototyping (free GPU if needed).	₹0	No paid Sentry; local suffices.

Stack Synergy: Node as "conductor"—e.g., Express route `/upload` → Supabase store → Queue Vosk → HF chain → MediaPipe → Supabase insert → Realtime push to React. Modular: Swap Vosk for Whisper later via config.

Integration Guide: From Planning to Build (Step-by-Step for Zero-Build Phase)

Since you're at "initial planning" (justice? assuming "just planning"), this is a phased

blueprint. Aim for **clarity checkpoints**—after each phase, sketch in a notebook/Draw.io. Total: 4 weeks to MVP demo.

Phase 1: Setup & Monorepo (Week 1, 5-7 hrs)

1. **Init Project**: `mkdir my-ai-coach && cd my-ai-coach && npx create-react-app frontend && mkdir backend && cd backend && npm init -y && npm i express multer fluent-ffmpeg bullmq @supabase/supabase-js .`
2. **Supabase Setup**: Sign up free (email only). Create project (Mumbai region). Enable Auth/Storage. In dashboard: Add table `videos` (id, user_id, url, insights JSON). Set RLS: `auth.uid() = user_id` .
3. **Env Config**: `.env` in backend: `SUPABASE_URL=your_url SUPABASE_ANON_KEY=your_key` . React: Install `@supabase/supabase-js` .
4. **Checkpoint**: Run `npm start` in both folders. Test Supabase upload via Postman (free): POST `/upload` with MP4.

Phase 2: Core Pipeline Integration (Weeks 2-3, 10-15 hrs)

1. **Video Upload Flow**: Backend Express: `app.post('/upload',`

`multer.single('video'), async (req, res) => { await supabase.storage.from('videos').upload(req.file.originalname, req.file.buffer); // Trigger queue }`.

2. **AI Orchestration**: Install AI libs: Backend `npm i vosk @xenova/transformers @mediapipe/tasks-vision python-shell` (for any Python bridges).
    - **STT Integration**: Download Vosk Hindi model (free ~50MB). `const vosk = require('vosk'); const model = new vosk.Model('./model'); const rec = new vosk.Recognizer(model, 16000); // Feed audio buffer from FFmpeg, get JSON transcript.
    - **Clarity/Tone/Structure**: `const { pipeline } = require('@xenova/transformers'); const summarizer = await pipeline('summarization', 'facebook/bart-large-cnn'); const sentiment = await pipeline('sentiment-analysis', 'tabularisai/multilingual-sentiment'); // Chain: transcript → sentiment/trans → summaries.
    - **Posture**: `const { PoseLandmarker } = require('@mediapipe/tasks-vision'); const vision = new PoseLandmarker({ /* config */ }); // Extract frames: fluent-ffmpeg(videoPath).screenshots({ timestamps: [5,10,...] }); vision.detectForVideo(frames); // Heuristics: if (hipAngle < 170) score -=1.
3. **Queueing**: `const Queue = require('bullmq').Queue; const analysisQueue = new Queue('analysis', { connection: { host: 'localhost', port: 6379 } }); // In-memory fallback. Add job: analysisQueue.add('process', { videoId });`.
4. **Gamification Hooks**: Supabase trigger (SQL in dashboard): `CREATE TRIGGER streak_update AFTER INSERT ON analyses FOR EACH ROW EXECUTE FUNCTION update_streak();`. React: `useEffect(() => { supabase.channel('insights').on('postgres_changes', ...).subscribe(); }` for realtime badges.
5. **Checkpoint**: End-to-end test: Record 1-min video in React → Upload → Local process (watch console) → See insights in Supabase → Dashboard refresh.

Phase 3: UI & Polish (Week 4, 5-7 hrs)

1. **React Integration**: Components: `<VideoRecorder />` (MediaRecorder), `<Gallery />`

(map over Supabase query: `supabase.from('videos').select('*')`), `<Dashboard />` (Recharts: <LineChart data={insights.confidence} />).

2. **Error Handling**: Wrap AI calls in try-catch; fallback: "Processing offline—retry in 5 min."
3. **Deploy Test**: `vercel --prod` for backend/frontend. Share link with 5 beta users (Google Form for wishlist).
4. **Checkpoint**: Full loop with 10 videos. Measure: Time <5 min? Accuracy >90% on samples?

Phase 4: Iteration (Ongoing)

- **Metrics**: Track via Supabase Analytics (free): Upload success rate.
- **Jugaad Optimizations**: Batch queues overnight (cron-job.org free); compress models if laptop heats (throttle frames).
- **Risks & Mitigations**: CPU overload? Limit to 1 video/user/day. Hindi accuracy dip? Fine-tune Vosk on free datasets (Kaggle).

Pros/Cons & Jugaad Wisdom

Pros:

- **Cost/Accessibility**: ₹0 entry; runs on any laptop (even 8GB RAM).
- **Speed to MVP**: Your skills + pre-built libs = 80% code reuse.
- **Future-Proof**: Modular (e.g., plug Azure later via env toggle).
- **Local-First**: No internet = reliable for MH blackouts.

Cons:

- **Performance**: CPU-bound (5 min/video vs. 30s on GPU)—mitigate with Colab for demos.
- **Accuracy Trade-Off**: 95% vs. 98% paid—test on your voice first.
- **Maintenance**: Model updates manual—set Google Alerts for HF releases.

Final Jugaad Tip: Like ISRO's "do more with less," start with a "Paper Prototype": Sketch

API flows on paper today. Join free communities (Reddit r/learnprogrammingIndia, Supabase Discord) for MH-specific tweaks. This stack is your Mangalyaan—launches big on pocket change.

Ready to build? Next: Phase 1 commands. Or refine (e.g., Hindi focus)? Bol, architect!

↳ Detailed integration code examples

↳ Hindi language model tweaks